

Process Synchronization

Process

A process is a sequentially program in execution.

Stack

This section contains local variable, functions and return address

Heap

This section is used to provide dynamic memory during runtime

Data

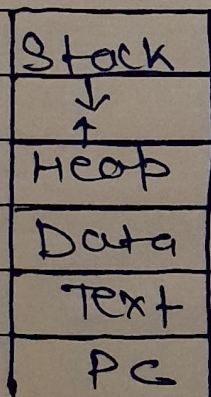
This section contains the global variables and static local variable

Text Section / ~~Program Counter~~

This section contains the executable instructions. It is only read only location, hence it.

Program Counter

Register in CPU that stores memory address of next instruction to be executed.



Process block,

Difference between Process and Program.

- | Process | Program |
|--|---|
| ① A process is a sequential program in execution. | A program is a set of instructions to perform task. |
| ② A process is entirely dependent on program | A program is independent |
| ③ Process is a module that executes concurrently | Program performs a task directly relating to an operation |
| ④ A process includes Program Counter, Text Section, Heap Sec., Data Sec., Stack Sec. | A program is just a set of instructions stored in a disk |
| ⑤ It is an active entity | It is a passive entity |

Co-operating Process

Co-operating processes are those processes which are affecting or affected by other processes executing in the system.

The benefit of having co-operating processes is

- ① Information Sharing
- ② Computation Speed up
- ③ modularity
- ④ Convenience

0 **IPC** Inter process Communication.
It is a model for communication b/w cooperating process.

There are 2 ways in IPC

- | ① Shared memory | ② message passing. |
|--|---------------------------------------|
| → Large amount of data | → Small amount of data |
| → Faster. | → Slower |
| → No kernel involvement as memory already in kernel. | → kernel involvement via system call. |

Producer Consumer problem.

Producer.

- ① Producer process produce data item.

A buffer is used between producer and consumer, Buffer size may be fixed or ~~variable~~ variable.

- ② The producer portion of the application generates data and stores it in the buffer.

- ③ The producer can't deposit its data if buffer is full.

Consumer.

- ① consumer process consumes data item.
- ② consumer reads the data stored in the buffer.
- ③ Consumer can't retrieve any data if the buffer is empty.

Producer - Consumer.

Struct :

item;

item buffer [Buffer_size]

int in;

int out;

int next producer;

while (True) {

/* produce an item and put in next produced */

while (Counter == Buffer_size);

// do nothing

CS for [buffer[in] = next producer;

producer in = (in+1) % Buffer_size;

Counter++;

}

int next consumer;

while (True) {

while (Counter == 0);

// do nothing

O.S for [next consumer = buffer[out];

Consumer. out = (out+1) % Buffer_size;

Counter--;

/* Consume the item assigned in the
next consumed */

}

Race Condition

A situation where several processes access and manipulate the same data concurrently and the outcome of the execution depends on a particular order in which access takes place is called Race condition.

To guard against Race Condition we need Synchronization.

Critical Section.. C.S

① A Critical Section is a code segment where all the processes are manipulating the shared data.

② An atomic action is required in a critical section i.e., only one process can execute in its critical section at a time.

All the other processes have to wait to execute in their critical section.

③ do {

 Entry Section // Entry Section handles the (critical section) entry into C.S

 Exit Section. // Exit Section handles (remainder section) // the exit from the C.S

} while (True);

④ Entry Section:-

It acquires resources needed for execution by the process.

Exit Section:-

It releases the resources and also informs other processes that the C.S is free.

⑤ Solution to the Critical Section Problem

① Mutual Exclusion :- An atomic action which ensures only one process can execute in C.S at a time

② Progress :-

Progress means ~~that~~ that only interested processes should be allowed in the C.S

③ Bounded wait :-

Bounded waiting means that each process must have a limited waiting time.

It should not wait endlessly to access the C.S

Above ~~are~~ solutions is used to prevent rare condition

Bakery Algorithm.

① It is used in multiple process solution.

② It solves the problem of C.S for n no. of processes.

③ Before entering the C.S, process receives a ticket number.

④ Holder of the smallest ticket no. ^{enters} ~~enters~~ the C.S

⑤ do {

 Choosing[i] = true; // process of choosing ticket no.

 number[i] = max(number[0], number[1], ...,

 number[n-1]) + 1;

 // ticket no. max of all currently chosen no.

 Choosing[i] = false; // finish choosing ticket no.

 for (j = 0; j < n; j++) {

 while (Choosing[j]); // wait if j is choosing.

 while ((number[j] != 0) && (number[j] < number[i]));

 // wait if j's number is smaller or j has the same no. but the smaller index

}

// critical section

number[i] = 0; // exit C.S.

// Remainder Section

} while (1);

Dekker's Algorithm

① Dekker's Algorithm is used in process queuing.

② It prevents the conflict by enforcing mutual exclusion with the use of flags and tokens.

③ The flag indicates the interests of a process.

④ The Tokens indicates priority, when both processes have their flags set to true.

Holder of the smallest token n.o. enters the C.S.

⑤ do {

wait (mutex);

// critical section

signal (mutex);

// remainder

} while (true);

Peterson's Solution.

- ① Peterson's algorithm is upgrade of Dekkers algo
- ② Peterson's algorithm is used for mutual exclusion and allows two processes to share a single-use resource without conflict.
- ③ It uses only shared memory for communication
- ④ There are two shared variables.
 - ① Turn. (int)
 - ② Flag. (boolean) binary.
- ⑤ The variable (Turn) indicates whose turn it is to enter the C.S.
The flag array is used to indicate if a process is ready to enter the C.S.
Flag[i] = true $\rightarrow$ implies that process P_i is ready.
Flag[i] = false $\rightarrow$ implies that process P_i is not ready.

Note:-

Peterson doesn't guarantee to work in modern computer architecture. $\therefore$ we use hardware solution.

- ⑥ do {
 Flag[i] = True;
 turn = i;
 While (Flag[j] && turn = j)
 // Critical Section
 Flag[i] = false;
 // remainder Section
} while(1)

Hardware Solution for Process Synchronization

① Test and Set.

Test and Set solution is a H/W solution which uses a h/w component called lock. This component ensures the mutual exclusion of the process.

Note:-

The implementation of this solution is difficult for application programmers $\therefore$ to overcome this difficulty semaphores are introduced.

② Semaphore

① Semaphores are integer variables that are used to solve the critical section problem by using two atomic operations, wait and signal that are used for process synchronization.

①. wait.

The wait operation decrements the value of its arguments (S), if it is positive.

If (S) is $\rightarrow$ ve or zero then no operation is performed.

wait (S)

{ while (S $\leq$ 0); //do nothing

S--;

}

②. Signal.

The Signal operation increments the value of its argument (S).

{ S++;

- ② The Semaphore guarantees the mutual exclusion, progress and in some cases bounded waiting.

Notes -

To overcome the bounded waiting condition Semaphore uses priority inheritance and the problem occurred is known as priority inversion.

- ③ There are 2 types of Semaphores.

① Counting Semaphore
which ranges from $(-\infty$ to $+\infty)$

② Binary Semaphore
which has 0 & 1 only.

Adv. of Semaphores.

- ① Semaphores allow only one process into the critical section. They follow mutual exclusion strictly.
- ② There is no resource wastage because of busy waiting in Semaphore.
- ③ Semaphores are implemented in microkernel. So they are m/c independent.

Disadvantages of Semaphore.

- ① Semaphores are complicated so the wait and signal operations must be implemented in the correct order, to prevent deadlocks.
- ② Semaphores may lead to priority inversion where low priority process may access the critical section first and higher priority wait.

Busy waiting. / Spin Lock

Busy waiting also known as Spin lock is a technique where a process repeatedly checks a condition until the busy bit becomes clear.

Pros :-

① Simple Implementation

② Cons :-

① Can cause → starvation/priority inversion or Deadlock.

Note :-

- ① To overcome busy waiting block function or suspended list. are used.
- ② To make a process sleep and Revert it by calling wakeup function

Classical problem of Synchronization

Page _____

Reader - Writer problem

① The reader writer problem relates to an object
Ex:- a file that is shared between multiple processes.

1a) Some of these processes are readers i.e., they only want to read data from object

1b) Some of these processes are writers i.e., they want to write into the object.

② The reader writer problem is used to manage Synchronization

2a) For ex:- If two readers access the object at the same time, there is no problem

2b) However if two writers or a reader writer access the object at the same time, there may be a problem.

③ To solve this problem, a writer should get exclusive access to an object i.e., when a writer is accessing the object, no reader or writer may access it.

3a) However multiple readers can access the object at the same time.

3b) Reader Process:

```
wait (mutex); // Enter C.S to update reader count.  
rc ++; // Increment the reader count  
if (rc == 1) // If this is the first reader.  
wait (wrt); // Request exclusive access to the  
Signal (mutex); // Shared Resource  
// Exit C.S to allow other  
// readers to update the reader  
// count.  
// C.S: Reading the Shared Resource.  
// Read
```

```
wait (mutex) // Enter C.S  
rc -- // Decr.  
if (rc == 0) // If this is the last reader  
Signal (wrt); // Release exclusive access to the  
Signal (mutex); // Shared Resource  
// Exit C.S
```

Writer process

```
wait (wrt); // Request exclusive access to the Shared  
// C.S: writing to the Shared Resource Resource  
// Write  
Signal (wrt); // Release exclusive access to the  
Shared Resource
```

Dining philosopher problem

① This problem states that there are 5 philosophers sitting on a circular table facing at the centre of the table. There are 5 chopsticks and one shared Rice bowl by using process synchronization, we have to ensure that no philosopher should starve.

② Circumstances

① only 2 operations a philosopher can do

① Eat

② Think

② To perform eat operation philosopher must have 2 chopsticks

③ To perform thinking operation philosopher must not have any chopsticks.

③ Solution

Chopsticks = 5

~~do i~~ do i

// Think

up (Chopstick[i]);

up (Chopstick[(i+1)%5]);

// Eat

down (Chopstick[i]);

down (Chopstick[(i+1)%5]);

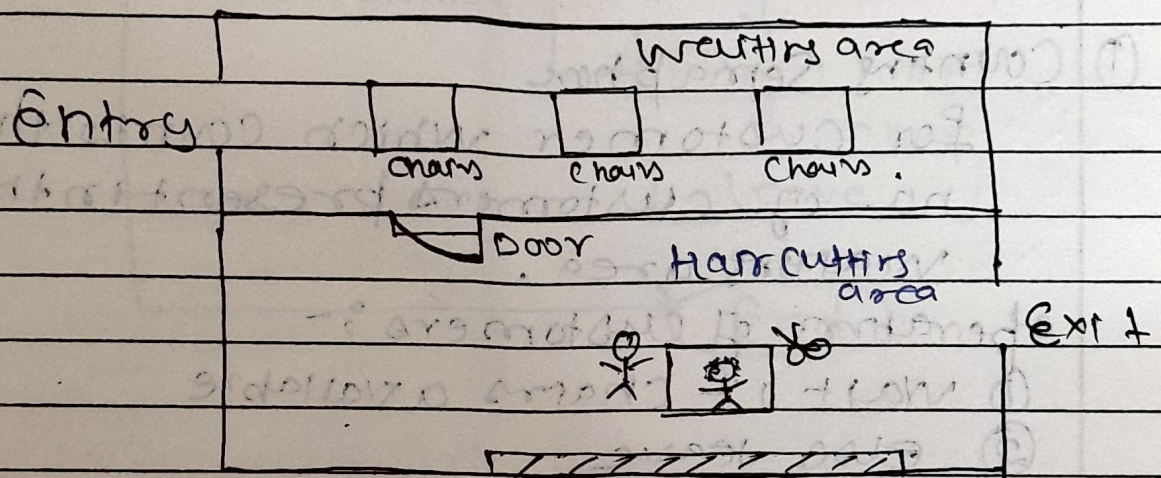
} while (True);

2 Semaphore phores are used in this solution.

① mutex is used such that no two philosophers can access the pickup or putdown at the same time.

② Semaphore array is used to control the behaviour of each philosopher.

Sleeping Barber problem



a) This analogy is based upon a hypothetical barber shop with one barber.

b) Situation:

1) one barber

2) one barber chair for hair cutting

3) n chairs in waiting area for customers

c) Operations:

1) If there are no customers, then barber sleeps on the barber chair.

2) When customer arrives in the hair cutting area, he has to wake up the barber.

3) If there are many customers and the barber is cutting a customer's hair, then the remaining customers either wait if there are empty chairs in the waiting area or they leave if no chairs are empty.

d) Solution

The solution to this problem includes 3 semaphores.

① Counting Semaphore

For customer which counts the no. of customers present in the waiting area.

operation of customers :-

- ① wait if chairs available
- ② else leave
- ③ get haircut.

② Binary Semaphore (mutex)

For Barber to tell whether barber is sleeping or haircutting with 0 and 1

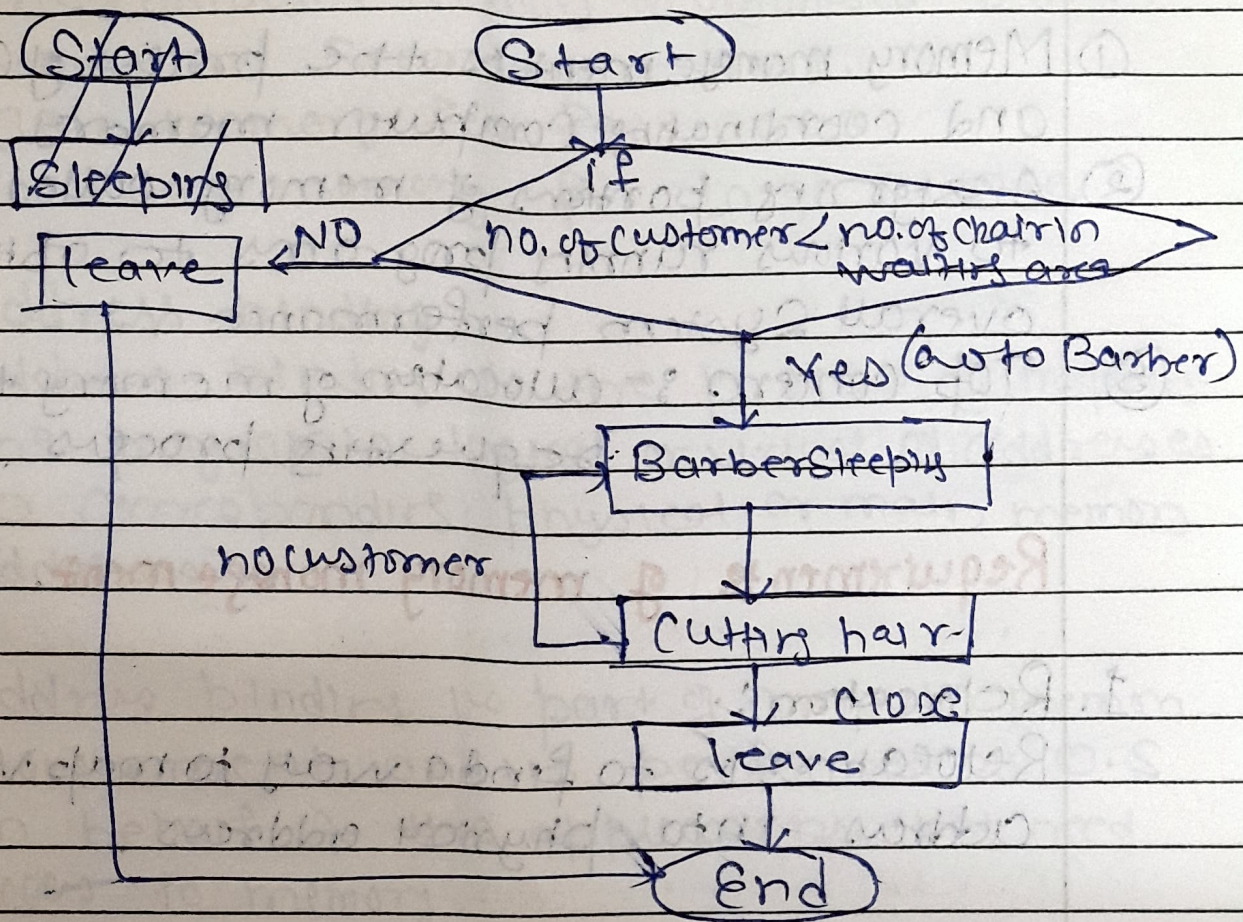
③ Binary Semaphore (mutex)

For providing mutual exclusion

→ Customer on Seat → 1

→ Customer Exit → 0

e) flow chart.



Memory Management

- ① Memory management is the process of controlling and coordinating computer memory,
- ② Assigning portions of memory called blocks to various running programs to optimize overall system performance.
- ③ Top concern :- allocation of memory to the requesting process.

Requirements of memory management.

1. Relocation:

Relocation is to find a way to map virtual address into physical address.

2. Sharing

Sharing is the flexibility to allow several processes to access the same portion of main memory.

3. Protection

Protection is to prevent a process from accessing memory that has ~~not~~ not been allocated to it.

4. Logical Organisation

Main Memory is organised as linear or one dimensional space that consists of bytes or words, similarly Secondary memory is organised at its physical level.